NOVA SCHOOL OF SCIENCE & TECHNOLOGY

DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

JOHN VERY LONGNAME DOE

Master/BSc in Name of Previous Degree

A VERY LONG AND IMPRESSIVE THESIS TITLE WITH A FORCED LINE BREAK

SOME THOUGHTS ON THE LIFE, THE UNIVERSE, AND EVERYTHING ELSE

MASTER IN AEROSPACE ENGINEERING

SPECIALIZATION IN SPECIALIZATION NAME

NOVA University Lisbon

Draft: *August 26, 2025*

NOVA SCHOOL OF SCIENCE & TECHNOLOGY

DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

A VERY LONG AND IMPRESSIVE THESIS TITLE WITH A FORCED LINE BREAK

SOME THOUGHTS ON THE LIFE, THE UNIVERSE, AND EVERYTHING ELSE

JOHN VERY LONGNAME DOE

Master/BSc in Name of Previous Degree

Adviser: Mary Doe Adviser Name
Full Professor, NOVA University Lisbon

Co-advisers: John Doe Co-Adviser Name
Associate Professor, NOVA University Lisbon

John Doe other Co-Adviser Name
Full Professor, NOVA University Lisbon

MASTER IN AEROSPACE ENGINEERING
SPECIALIZATION IN SPECIALIZATION NAME

NOVA University Lisbon
Draft: *August 26, 2025*

Abstract

Resumo

Contents

List of Figures v

List of Algorithms vi

Acronyms vii

1 Introduction 1

2 Methodology 2

2.1 CLF-CBF . . . 2

2.1.1 First-Order Formulation . . . 2

2.1.2 Backstepping (Second-Order Formulation) . . . 5

2.1.3 Closed Form . . . 6

2.1.4 Unifying CBF . . . 8

2.2 Single Parameter Optmization . . . 12

2.2.1 Gradient of the Cost Function - Discretization . . . 12

2.2.2 Gradient Descent . . . 12

2.2.3 Binary Search . . . 13

2.3 Constraint Contour Point . . . 15

2.3.1 Constraint Fit . . . 15

2.3.2 State Propagation Simulation . . . 15

2.3.3 Constraint Contour Point Algorithm . . . 18

2.4 Proposed Decision Algorithms . . . 20

2.4.1 Just Optimized (A-JO) . . . 20

2.4.2 Double CLF . . . 21

3 Results 23

3.1 Setup . . . 23

3.1.1 Decision Algorithms . . . 23

3.1.2 Unicycle . . . 24

3.1.3 Orbital . . . 26

3.1.4 Some Data . . . 26

3.2 Experiments . . . 26

4 Conclusion **27**

Bibliography **28**

Appendices

A Deductions **29**

A.1 Closed-Form QP given CLF-CBF Constraints 29

A.2 Closed-Form Ellipsoide Tangent Point 33

A.3 Multiple CLF Transition Formulation 33

List of Figures

List of Algorithms

1 Discrete Gradient Function (DGF) 12
2 Binary Search Phase 1 14
3 Binary Search Phase 2 14
4 State Propagation Simulation (SPS) 17
5 Natural Convergence Direction Vector ($NCDV$) 18

Acronyms

A-CLF-I CLFs Independent *(pp. 22–24)*
A-CLF-S CLFs Summed *(pp. 21, 23, 24)*
A-CLF-U CLFs Unified *(p. 22)*
A-JO Just Optimized *(pp. iii, 20, 22, 23)*

CBF Control Barrier Function *(pp. iii, 3–11, 20–22, 25, 26, 29–31)*
CLF Control Lyapunov Function *(pp. iii, iv, 2–7, 11, 20–22, 24–26, 29–33)*
CLF-CBF Control Lyapunov Function - Control Barrier Function *(pp. iii, iv, 2, 3, 5, 7–9, 11, 15, 20, 21, 24, 26, 29, 31, 33)*

DP Dormand Prince *(p. 26)*

KKT Karush-Kuhn-Tucker *(pp. 7, 29, 31, 32)*

ODE ordinarinary differential equation *(p. 15)*

QP Quadratic program *(pp. iv, 4–8, 11, 20, 21, 29, 31, 33)*

RK Runge-Kutta *(pp. 15–17, 20, 21)*

1

Introduction

2

Methodology

2.1 CLF-CBF

Considering a nonlinear control-affine system:

$$\dot{\mathbf{x}} = f(\mathbf{x}) + g(\mathbf{x})\mathbf{u} \tag{2.1}$$

with stet $\mathbf{x} \in \mathbb{R}^n$, control input $\mathbf{u} \in \mathbb{R}^m$, and locally Lipschitz continuos[1] functions $f : \mathbb{R}^n \to \mathbb{R}^n$ and $g : \mathbb{R}^n \to \mathbb{R}^{n\times m}$. Given a locally Lipschitz continuos $k : \mathbb{R}^n \to \mathbb{R}^m$, it results in the closed-loop system:

$$\dot{\mathbf{x}} = f(\mathbf{x}) + g(\mathbf{x})k(\mathbf{x}) \tag{2.2}$$

For any initial condition $\mathbf{x}_0 \in \mathbb{R}^n$, there is a maximal time interval $\mathcal{I}(\mathbf{x}_0) = [0; t_{max}(\mathbf{x}_0)[$ and unique continuosly differentiable solution $\varphi : \mathcal{I}(\mathbf{x}_0) \to \mathbb{R}^n$ such that:

$$\begin{aligned} &\dot{\varphi}(t) = f(\varphi(t)) + g(\varphi(t))k(\varphi(t)) \quad \forall t \in \mathcal{I}(\mathbf{x}_0) \\ &\varphi(0) = \mathbf{x}_0 \end{aligned} \tag{2.3}$$

2.1.1 First-Order Formulation

2.1.1.1 Control Lyapunov Function (CLF)

Having the objective of global assymptotically stability of the system 2.2, is desired for the system converge to an equilibrium point $\bar{\mathbf{x}}$. A solution comes by designing a positive definite function $V : \mathbb{R}^n \to \mathbb{R}_{\geq 0}$ which converges to zero while the system converges to the respective equilibrium point, and for that, designing a control law that acts in order to achieve specifically the convergence of the function. Consequentially is pretended to the function $V(\mathbf{x})$ to be continuos and differentiable:

$$\dot{V}(\mathbf{x}, \mathbf{u}) = \frac{\partial V}{\partial x}\dot{x} = L_f V(\mathbf{x}) + L_G V(\mathbf{x})\mathbf{u}$$

where $L_f V$ and $L_G V$ are the Lie derivatives of V and $\mathbf{u}$ the control input vector resposible to manipulate the function, more precisely, its convergence rate.

[1] Given two metric spaces (X, d_X) and (Y, d_Y), where d_X and d_Y represent the X and Y metrics respectively, a function $f : X \to Y$ is Lipschitz continuos if there exist a constant $L \geq 0$ such that $\forall (x_1, x_2) \in X, \quad d_Y(f(x_1), f(x_2)) \leq L\, d_X(x_1, x_2)$

That beeing said, a continuosly differentiable, positive definite function $V : \mathbb{R}^n \to \mathbb{R}_{\geq 0}$ wich $\lim_{\mathbf{x}\to\bar{\mathbf{x}}} V(\mathbf{x}) = 0$ is a CLF if there exists a $\gamma \in \mathcal{K}_\infty^e$ [2] such that for all $\mathbf{x} \in \mathbb{R}^n \setminus \{\bar{\mathbf{x}}\}$:

$$\inf_{\mathbf{u}\in\mathbb{R}^m} [L_f V(\mathbf{x}) + L_G V(\mathbf{x})\mathbf{u}] \leq -\gamma(V(\mathbf{x})) \tag{2.4}$$

From CLF V and an adequate tuning of γ parameter results a control law followed by the controller K_{CLF}:

$$K_{CLF}(\mathbf{x}) = \{\mathbf{u} \in \mathbb{R}^m : L_f V(\mathbf{x}) + L_G V(\mathbf{x})\mathbf{u} \leq -\gamma(V(\mathbf{x}))\} \tag{2.5}$$

The stets generated by $K_{CLF}(\mathbf{x})$ $\forall x \in \mathbb{R}^n$ refer to the possible control input values necessary to the CLF $V(\mathbf{x})$ continuosly converge to zero while the closed-loop system 2.2 to the equilibrium point, but more slowly as is close to reach the objective.

2.1.1.2 Control Barrier Function (CBF)

Now having the objective of garantee safety to system 2.2, is aimed to keep the state inside a safe set C (foward invariant[3]) defined accordingly the problem needs. The notion of safety, i.e., the safe set C is defined as the 0-superlevel[4] set of a continuosly differentiable function $h : \mathbb{R}^n \to \mathbb{R}$:

$$\begin{aligned} C &= \{\mathbf{x} \in \mathbb{R}^n : h(\mathbf{x}) \geq 0\} \\ \partial C &= \{\mathbf{x} \in \mathbb{R}^n : h(\mathbf{x}) = 0\} \\ int(C) &= \{\mathbf{x} \in \mathbb{R}^n : h(\mathbf{x}) > 0\} \end{aligned} \tag{2.6}$$

In order to mantain $h(\mathbf{x})$ positive, there is a need to have a control law and but first to be able to interface with the function. Since $h(\mathbf{x})$ is differentiable:

$$\dot{h}(\mathbf{x}, \mathbf{u}) = \frac{\partial h}{\partial x}\dot{x} = L_f h(\mathbf{x}) + L_G h(\mathbf{x})\mathbf{u}$$

where $L_f h$ and $L_G h$ are the Lie derivatives of h and $\mathbf{u}$ the control input vector resposible to manipulate the function, more precisely, its time derivative.

So, a differentiable function $h : \mathbb{R}^n \to \mathbb{R}$ is a CBF if there exists a $\alpha \in \mathcal{K}_\infty^e$ such that for all $\mathbf{x} \in \mathbb{R}^n \setminus \{\bar{\mathbf{x}}\}$:

$$\sup_{\mathbf{u}\in\mathbb{R}^m} [L_f h(\mathbf{x}) + L_G h(\mathbf{x})\mathbf{u}] \geq -\alpha(h(\mathbf{x})) \tag{2.7}$$

Similar to the CLF, from CBF h and an adequate tuning of α parameter results a control law encapsulated by the controller K_{CBF}:

$$K_{CBF}(\mathbf{x}) = \{\mathbf{u} \in \mathbb{R}^m : L_f h(\mathbf{x}) + L_G h(\mathbf{x})\mathbf{u} \geq -\alpha(h(\mathbf{x}))\} \tag{2.8}$$

[2] If a continuos functions $\alpha : [-b, a] \to \mathbb{R}$ with $(a, b) > 0$, strictly increasing and $\alpha(0) = 0$ then $\alpha \in \mathcal{K}^e$ and if it adds $\lim_{r\to\infty} \alpha(r) = \infty$ then $\alpha \in \mathcal{K}_\infty^e$

[3] A set $\mathcal{S}$ is said foward invariant if the initial system state $x_0 \in \mathcal{S}$ and so $x(t) \in \mathcal{S}$ $\forall t \geq 0$

[4] For function $f : \mathbb{R}^n \to \mathbb{R}$, its superlevel set is given by $\mathcal{S}(f) = \{(x_1, ..., x_n) : f(x_1, ..., x_n) \geq c\}$, where having $c = 0$ corresponds to the 0-superlevel set of f

The controller $K_{CLF}(\mathbf{x})\ \ \forall x \in \mathbb{R}^n$ renders control input sets that keep the closed-loop system 2.2 safe with respect to the set C. The controller by defenition is able to evolve the closed-loop system more freely when $\dot{h}(\mathbf{x}) > 0$ and more cautiously as $h(\mathbf{x})$ shortens and $\dot{h}(\mathbf{x}) < 0$, if $h(\mathbf{x}) < 0$ it means is in the unsafe region, so $\dot{h}(\mathbf{x}) > 0$ implies converging continuosly to the safe set C.

2.1.1.3 Quadratic Program Formulation

Now having the objective of not only achieve safety but also assymptotically stability through the usage of CLF and CBF it renders a set of control inputs able to achieve the respective purposes.

Using the conditions imposed by CLF and CBF as constraints, is presented an optimization problem via Quadratic program (QP) [5], given a locally Lipschitz continuos controller $k : \mathbb{R}^n \longrightarrow \mathbb{R}^m$:

$$\begin{aligned} k(\mathbf{x}) = \underset{\mathbf{u}\in\mathbb{R}^m}{\operatorname{argmin}}\ & \tfrac{1}{2}(\mathbf{u}^\top R\mathbf{u}) \\ st.\ & L_f V(\mathbf{x}) + L_G V(\mathbf{x})\mathbf{u} \leq -\gamma(V(\mathbf{x})) \\ & L_f h(\mathbf{x}) + L_G h(\mathbf{x})\mathbf{u} \geq -\alpha(h(\mathbf{x})) \end{aligned} \tag{2.9}$$

The sets of control inputs restricted to the optimization problem are given by $K_{CLF-CBF}(\mathbf{x}) = K_{CLF}(\mathbf{x}) \cap K_{CBF}(\mathbf{x})$ hopefully non-empty which can happen if the conditions are in conflict.

In case of conditions' conflict, in order to the closed loop system 2.2 controller have a solution from the QP, since safety is critical, using a slack variable δ allows to relax the stability condition imposed by the CLF and highly contribute to obtaining a solution while keeping system safe:

$$\begin{aligned} k(\mathbf{x}) = \underset{\mathbf{u}\in\mathbb{R}^m,\ \delta\in\mathbb{R}}{\operatorname{argmin}}\ & \tfrac{1}{2}(\mathbf{u}^\top R\mathbf{u} + p\delta^2) \\ st.\ & L_f V(\mathbf{x}) + L_G V(\mathbf{x})\mathbf{u} \leq -\gamma(V(\mathbf{x})) + \delta \\ & L_f h(\mathbf{x}) + L_G h(\mathbf{x})\mathbf{u} \geq -\alpha(h(\mathbf{x})) \end{aligned} \tag{2.10}$$

Having δ as an optimization variable not only allows to balance between the input values exceed usage and the level of relaxation but also to not have a fixed δ value which would result for all states not so faster convergence and to the neighborhood of the equilibrium point instead of the actual target point.

Notes:

- If it's not pretended to enforce stability or safety, just remove from the optimization problem the respective constraint responsible to it

[5] Quadratic program consists of solving a mathematical optimization problem with a quadratic objective function

2.1.2 Backstepping (Second-Order Formulation)

2.1.2.1 Second-Order Systems

Usually control systems are governed by higher-order systems, such as some second-order cyber-physical systems controlling the position by applying some force to manipulate the speed that is felt in the position just instants after. Higher order systems' outputs variation is felt by an multiple integrative effect, by an integration of each state affecting the next state dynamic until reach the so wanted output. Considering a non-linear system in strict-feedback form:

$$\dot{\mathbf{x}} = f_0(\mathbf{x}) + g_0(\mathbf{x})\xi \tag{2.11a}$$

$$\dot{\xi} = f_1(\mathbf{x}, \xi) + g_1(\mathbf{x}, \xi)\mathbf{u} \tag{2.11b}$$

with $\mathbf{x} \in \mathbb{R}^n$, $\xi \in \mathbb{R}^p$, and $\mathbf{u} \in \mathbb{R}^m$, and functions $f_0 : \mathbb{R}^n \to \mathbb{R}^n$, $g_0 : \mathbb{R}^n \to \mathbb{R}^{n\times p}$, $f_1 : \mathbb{R}^n \times \mathbb{R}^p \to \mathbb{R}^p$ and $g_1 : \mathbb{R}^n \times \mathbb{R}^p \to \mathbb{R}^{p\times m}$ locally Lipschitz continuos on their respective domains.

Given a locally Lipschitz continuos feedback controller $\mathbf{k} : \mathbb{R}^n \times \mathbb{R}^p \to \mathbb{R}^m$ the closed-loop system is:

$$\begin{aligned} \dot{\mathbf{x}} &= f_0(\mathbf{x}) + g_0(\mathbf{x})\xi \\ \dot{\xi} &= f_1(\mathbf{x}, \xi) + g_1(\mathbf{x}, \xi)\mathbf{k}(\mathbf{x}, \xi) \end{aligned} \tag{2.12}$$

for any initial states $(\mathbf{x}_0, \xi_0) \in \mathbb{R}^n \times \mathbb{R}^p$ exists a maximum time interval $I(\mathbf{x}_0, \xi_0) \subseteq \mathbb{R}_{\geq 0}$ and a unique solution $\varphi = (\varphi_{\mathbf{x}}, \varphi_{\xi})$ satisfying 2.3 $\forall t \in I(\mathbf{x}_0, \xi_0)$.

2.1.2.2 Purposal Solution Obtaining

Since having a second order system 2.11 the access to ξ is not direct, we must "backstepp" in order to be able to control $\mathbf{u}$, affect ξ and consequentially control the wanted dynamic $\dot{\mathbf{x}}$. The strategy passes by obtaining the solution if it was a first order system 2.1 and if ξ could be directly controlled (i.e. as part of the first order system control input), so given a differentiable and locally Lipschitz continuos $k_0(\mathbf{x}) : \mathbb{R}^n \to \mathbb{R}^p$ (that encompasses the "controllable ξ"), the closed-loop system:

$$\dot{\mathbf{x}} = f_0(\mathbf{x}) + g_0(\mathbf{x})k_0(\mathbf{x}) \tag{2.13}$$

That beeing said, the first order solution could be obtained via QP, so supposing a CLF V_0, CBF h_0, a differentiable and locally Lipschitz continuos described just now $k_0(\mathbf{x})$ and $(\gamma_0, \alpha_0) \in \mathcal{K}^e_\infty \times \mathcal{K}^e_\infty$:

$$\begin{aligned} k_0(\mathbf{x}) = \underset{\mathbf{u}\in\mathbb{R}^p,\ \delta\in\mathbb{R}}{\operatorname{argmin}} \ & \tfrac{1}{2}(\mathbf{u}^\top R_0 \mathbf{u} + p_0\delta^2) \\ st.\ \ & L_f V_0(\mathbf{x}) + L_G V_0(\mathbf{x})\mathbf{u} \leq -\gamma_0(V_0(\mathbf{x})) + \delta \\ & L_f h_0(\mathbf{x}) + L_G h_0(\mathbf{x})\mathbf{u} \geq -\alpha_0(h_0(\mathbf{x})) \end{aligned} \tag{2.14}$$

The solution is equivalent to a value of ξ closest to an optimal value taken by it that would impose safety and contribute to stability not only to the first-order system 2.13 but also to the actual second-order system. Although like it was said is not possible to have direct access to ξ, the backstepping technique is essentially the second-order trying to converge to the first order solution.

Now considering the second-order system 2.11, supposing there is $V_\xi(\mathbf{x}, \xi) : \mathbb{R}^n \times \mathbb{R}^p \to \mathbb{R}_{\geq 0}$, the new CLF $V(\mathbf{x}, \xi)$ and CBF $h(\mathbf{x}, \xi)$ are:

$$V_\xi(\mathbf{x}, \xi) = \frac{1}{2}\|\xi - k_0(\mathbf{x})\|^2 \tag{2.15a}$$

$$V(\mathbf{x}, \xi) = V_0(\mathbf{x}) + \eta V_\xi(\mathbf{x}, \xi) \tag{2.15b}$$

$$h(\mathbf{x}, \xi) = h_0(\mathbf{x}) - \eta V_\xi(\mathbf{x}, \xi) \tag{2.15c}$$

With $\eta \in \mathbb{R}_{\geq 0}$. Finally, via QP, given a differentiable and locally Lipschitz continuos $k(\mathbf{x}, \xi) : \mathbb{R}^n \times \mathbb{R}^p \to \mathbb{R}^m$:

$$\begin{aligned} k(\mathbf{x}, \xi) = \underset{\mathbf{u}\in\mathbb{R}^m,\, \delta\in\mathbb{R}}{\operatorname{argmin}} \ & \tfrac{1}{2}(\mathbf{u}^\top R\mathbf{u} + p\delta^2) \\ st. \ & L_f V(\mathbf{x}, \xi) + L_G V(\mathbf{x}, \xi)\mathbf{u} \leq -\gamma(V_0(\mathbf{x})) - \gamma'(V_\xi(\mathbf{x}, \xi)) + \delta \\ & L_f h(\mathbf{x}, \xi) + L_G h(\mathbf{x}, \xi)\mathbf{u} \geq -\alpha(h_0(\mathbf{x})) + \alpha'(V_\xi(\mathbf{x}, \xi)) \end{aligned} \tag{2.16}$$

It's expected that $\gamma << \gamma'$ contributing to a faster convergence of ξ and more similar behaviour of the system 2.12 relative to the first-order system 2.13 and mitigate the chances of the system 2.12 not be safe in relation to C_0.

Notes:

- If it's not pretended to enforce stability or safety, just remove from both optimization problem (2.14 and 2.16) the respective constraint responsible to it
- If it's a higher-order (than second) system, then, assuming it's a third-order system, the solution obtained in 2.16 is the "new k_0" and the third-order solution tries to converge to the second-order one (while this one is converging to the first-order solution)

2.1.3 Closed Form

The objective of this thesis is to purpose a set of low computational effort techniques suitable to spacecrafts. Aiming a faster computation of the optimization problems solutions, any of the previous QPs (2.9, 2.10, 2.14, 2.16) have a closed-form[6] solution which allows to calculate directly the respective solution instead of depending of a numerical solver. Given the QP optimization problem and a controller $k(\mathbf{x}, \xi) : \mathbb{R}^n \times \mathbb{R}^p \to \mathbb{R}^m$ (if it is a second-order system):

$$\begin{aligned} k(\mathbf{x}, \xi) = \underset{\mathbf{u}\in\mathbb{R}^m,\, \delta\in\mathbb{R}}{\operatorname{argmin}} \ & \tfrac{1}{2}(\mathbf{u}^\top R\mathbf{u} + p\delta^2) \\ st. \ & \underbrace{L_f V(\mathbf{x}, \xi) + \gamma(V_0(\mathbf{x})) + \gamma'(V_\xi(\mathbf{x}, \xi))}_{F_V(\mathbf{x},\xi)} + L_G V(\mathbf{x}, \xi)\mathbf{u} - \delta \leq 0 \\ & \underbrace{L_f h(\mathbf{x}, \xi) + \alpha(h_0(\mathbf{x})) - \alpha'(V_\xi(\mathbf{x}, \xi))}_{F_h(\mathbf{x},\xi)} + L_G h(\mathbf{x}, \xi)\mathbf{u} \geq 0 \end{aligned} \tag{2.17}$$

[6] An closed-form expression is expressed by a limit number and well defined operations (elementary functions)

With a CLF V, CBF h, $\mathbf{x} \in \mathbb{R}^n$, the first-order system CLF V_0(= V like in 2.9, 2.10 and 2.14) , and if it is a second-order system with $\xi \in \mathbb{R}^p$ (equivalent to the QP seen while doing Backstepping 2.16).

Using [3] as a reference, assuming equal control input wheights relative to each other ($R = \mathbf{I}_{m\times m}$) and just one CLF and CBF each, according to the Karush-Kuhn-Tucker (KKT) conditions[7] (to see the full deduction A.1):

$$\mathbf{k}(\mathbf{x}, \xi) = \begin{cases} \mathbf{k}_1(\mathbf{x}, \xi), & (\mathbf{x}, \xi) \in \mathcal{S}_1 \\ \mathbf{k}_2(\mathbf{x}, \xi), & (\mathbf{x}, \xi) \in \mathcal{S}_2 \\ \mathbf{k}_3(\mathbf{x}, \xi), & (\mathbf{x}, \xi) \in \mathcal{S}_3 \\ \mathbf{0}_{m\times 1} \quad , & (\mathbf{x}, \xi) \in \mathcal{S}_4 \end{cases} \tag{2.18}$$

Where each controller $\mathbf{k}_i$, for $i = 1, ..., 4$, corresponds to the closed-form equation depending which constraints (CLF or CBF) are active what can be inferred by $\mathcal{S}_i$, for $i = 1, ..., 4$. Via KKT conditions the respectives controllers and domains are obtained due to the KKT multipliers.

Auxiliary Variables:

$$\begin{aligned} L(\mathbf{x}, \xi) &= L_G V(\mathbf{x}, \xi)\, L_G h(\mathbf{x}, \xi)^\top \\ \Delta &= L(\mathbf{x}, \xi)^2 (p^{-1} + \|L_G h(\mathbf{x}, \xi)\|^2)\, \|L_G h(\mathbf{x}, \xi)\|^2 \end{aligned}$$

KKT multipliers:

$$\begin{aligned} \lambda_1(\mathbf{x}, \xi) &= \Delta^{-1}(L(\mathbf{x}, \xi)\, F_h(\mathbf{x}, \xi) \; - \|L_G h(\mathbf{x}, \xi)\|^2\, F_V(\mathbf{x}, \xi)) \\ \lambda_2(\mathbf{x}, \xi) &= \Delta^{-1}(L(\mathbf{x}, \xi) F_V(\mathbf{x}, \xi) \; - (p^{-1} + \|L_G h(\mathbf{x}, \xi)\|^2)\, F_h(\mathbf{x}, \xi)) \end{aligned} \tag{2.19}$$

Controller $\mathbf{k}_i$ closed-form equation:

$$\begin{aligned} \mathbf{k}_1(\mathbf{x}, \xi) &= -(p^{-1} + \|L_G V(\mathbf{x}, \xi)\|^2)^{-1}\, F_V(\mathbf{x}, \xi)\, L_G V(\mathbf{x}, \xi)^\top \\ \mathbf{k}_2(\mathbf{x}, \xi) &= -\|L_G h(\mathbf{x}, \xi)\|^{-2}\, F_h(\mathbf{x}, \xi)\, L_G h(\mathbf{x}, \xi)^\top \\ \mathbf{k}_3(\mathbf{x}, \xi) &= -\lambda_1(\mathbf{x}, \xi) L_G V(\mathbf{x}, \xi)^\top + \lambda_2(\mathbf{x}, \xi) L_G h(\mathbf{x}, \xi)^\top \end{aligned} \tag{2.20}$$

Constraint Activation Subdomains $\mathcal{S}_i$ (based on the dual feasibilitie condition [7]):

If CLF is active:

$$\mathcal{S}_1(\mathbf{x}, \xi) = \{(\mathbf{x}, \xi) \in \mathbb{R}^n \times \mathbb{R}^p : \lambda_1 \geq 0 \cap \lambda_2 < 0\}$$

If CBF is active:

$$\mathcal{S}_2(\mathbf{x}, \xi) = \{(\mathbf{x}, \xi) \in \mathbb{R}^n \times \mathbb{R}^p : \lambda_1 < 0 \cap \lambda_2 \geq 0\} \tag{2.21}$$

[7] KKT Conditions Wikipedia (Click here to be Redirected)

If CLF and CBF is active:

$$\mathcal{S}_3(\mathbf{x}, \xi) = \{(\mathbf{x}, \xi) \in \mathbb{R}^n \times \mathbb{R}^p : \lambda_1 \geq 0 \cap \lambda_2 \geq 0\}$$

If Any is active:

$$\mathcal{S}_4(\mathbf{x}, \xi) = \{(\mathbf{x}, \xi) \in \mathbb{R}^n \times \mathbb{R}^p : \lambda_1 < 0 \cap \lambda_2 < 0\}$$

2.1.4 Unifying CBF

The previous controllers synthesized control inputs based on just one CBF, i.e., those controllers garantee safety w.r.t. a single safe set $\mathcal{C}$, but given diffrent scenarios, it could exist more complex safety specifications and the need to impose more than one safety contraints simultaneously such as CBF $h_i(\mathbf{x})$, for $i \in \mathcal{I} = \{1, ...N\}$:

$$\mathcal{C}_i = \{\mathbf{x} \in \mathbb{R}_n : h_i(\mathbf{x}) \geq 0\} \tag{2.22}$$

$$\mathcal{C} = \bigcap_{i \in \mathcal{I}} \mathcal{C}_i \tag{2.23}$$

But like it was said before regarding having more than one constraint could leave to conflict problems, empty safe sets ($\mathcal{C} = \emptyset$) and an inexistent set of acceptable control inputs ($\mathbf{k}(\mathbf{x}, \xi) = \emptyset$). Using [4] as guidance, in order to avoid conflicts, merges all CBF into a single one that is able to capture all those safety notions and allow for even more complex safety specifications and to the resultant constraint fit the already obtained CLF-CBF closed-form solution 2.18.

2.1.4.1 First-Order System

The more complex safety specifications are defined via boolean logical operations between 0-superlevel sets of the diffrent CBF. Those operations are tipically seen in constrained optimization problems, like in QP of the form:

$$\begin{aligned} k(\mathbf{x}) = \underset{\mathbf{u} \in \mathbb{R}^p,\, \delta \in \mathbb{R}}{\operatorname{argmin}} \ & \tfrac{1}{2}(\mathbf{u}^\top R \mathbf{u} + p\delta^2) \\ st. \ & L_f V(\mathbf{x}) + L_G V(\mathbf{x})\mathbf{u} \leq -\gamma(V(\mathbf{x})) + \delta \\ & L_f h_i(\mathbf{x}) + L_G h_i(\mathbf{x})\mathbf{u} \geq -\alpha_i(h_i(\mathbf{x})) \quad \forall i \in \mathcal{I} \end{aligned} \tag{2.24}$$

Where the resultant safe set 2.23 of the system refers to the intersection of all safe sets 2.22. More than be propitious to be not solvable, the formulation itself of the optimization problem 2.24 is limitated by no other logical operation than AND.

Among the the logical operations that could be applied to CBFs, there are:

Identity (Class-$\mathcal{K}^e$):

$$\mathcal{C}_i = \{(\mathbf{x}, \alpha_i) \in \mathbb{R}^n \times \mathcal{K}^e : \alpha_i(h_i(\mathbf{x}) \geq 0)\} = \tag{2.25a}$$

$$= \{\mathbf{x} \in \mathbb{R}^n : h_i(\mathbf{x}) \geq 0\} \tag{2.25b}$$

Complement Set (Negation):

$$\bar{C}_i = \{\mathbf{x} \in \mathbb{R}^n : -h_i(\mathbf{x}) \geq 0\} \tag{2.26}$$

Intersection of Sets (AND):

$$\begin{aligned} \bigcap_{i \in \mathcal{I}} C_i &= \{\mathbf{x} \in \mathbb{R}^n : h_i(\mathbf{x}) \geq 0 \ \ \forall i \in \mathcal{I}\} \\ &= \{\mathbf{x} \in \mathbb{R}^n : \min_{i \in \mathcal{I}}(h_i(\mathbf{x})) \geq 0\} \end{aligned} \tag{2.27}$$

Union of Sets (OR):

$$\begin{aligned} \bigcup_{i \in \mathcal{I}} C_i &= \{\mathbf{x} \in \mathbb{R}^n : \exists i \in \mathcal{I} \ \ st. \ \ h_i(\mathbf{x}) \geq 0\} \\ &= \{\mathbf{x} \in \mathbb{R}^n : \max_{i \in \mathcal{I}}(h_i(\mathbf{x})) \geq 0\} \end{aligned} \tag{2.28}$$

The logical operations identity, complement, intersection and union are represented by respectively, class-$\mathcal{K}^e$ functions, negation, min and max operations. Although the class-$\mathcal{K}^e$ ($\alpha_i(h_i(\mathbf{x}))$) and negation ($-h_i(\mathbf{x})$) functions which are continuosly differentiable and valid as CBF, contrary to the AND and OR functions, $\min_{i \in \mathcal{I}}(h_i(\mathbf{x}))$ and $\max_{i \in \mathcal{I}}(h_i(\mathbf{x}))$ that aren't continuosly differentiable and therefore not CBF candidates. The max and min functions capacitate combining safe sets defined by multiple CBF (accordingly to the operation) and to obtain a new safety specification as is expected to see while merging multiple CBF into a single one. So, in order to viablize the AND and OR operation and achieve the merge of CBFs, the min and max functions can be replaced via smooth-aproximation, keeping their properties with just a relative small degradation of accuracy, by continuosly differentiable **log-sum-exp functions**:

Intersection of Sets \AND \ Minimum

$$\begin{aligned} h(\mathbf{x}) &= \frac{-1}{k} ln(\sum_{i \in \mathcal{I}} e^{-k\ h_i(\mathbf{x})}) \\ &\approx \min_{i \in \mathcal{I}}(h_i(\mathbf{x})) \end{aligned} \tag{2.29}$$

Beeing $k \in \mathbb{R}_{>0}$ as smoothing parameter (if $k < 0$ then is equivalent to the smooth-aproximation of $\max_{i \in \mathcal{I}}(h_i(\mathbf{x}))$).

$$\dot{h}(\mathbf{x}, \mathbf{u}) = \underbrace{\sum_{i \in \mathcal{I}} [\lambda_i \ L_f h_i(\mathbf{x})]}_{Lfh(\mathbf{x})} + \underbrace{(\sum_{i \in \mathcal{I}} [\lambda_i \ L_G h_i(\mathbf{x})])}_{LGh(\mathbf{x})}\mathbf{u} \tag{2.30a}$$

$$\lambda_i(\mathbf{x}) = e^{-k(h_i(\mathbf{x}) - h(\mathbf{x}))} \tag{2.30b}$$

With $\sum_{i \in \mathcal{I}} \lambda_i(\mathbf{x}) = 1$

Encountering an under-aproximation of h relative to the min function, given the safe set C defined based on 2.6 it verifies:

$$\min_{i \in \mathcal{I}}(h_i(\mathbf{x})) \geq h(\mathbf{x}) \quad \forall \mathbf{x} \in \mathbb{R}^n \tag{2.31a}$$

$$C \subseteq \bigcap_{i \in \mathcal{I}} C_i \tag{2.31b}$$

But, $\lim_{k\to\infty} h(\mathbf{x}) = \min_{i\in\mathcal{I}}(h_i(\mathbf{x}))$ and $\lim_{k\to\infty} C = \bigcap_{i\in\mathcal{I}} C_i$

On par with that, the new CBF $h(\mathbf{x})$ 2.29 could restrain too much the closed-loop system in comparison to theoretical CBF $\lim_{k\to\infty} h(\mathbf{x})$. In order to relax the safe set C and potentially contain $\bigcap_{i\in\mathcal{I}} C_i$:

$$h(\mathbf{x}) = \frac{-1}{k} ln(\sum_{i\in\mathcal{I}} e^{-k\ h_i(\mathbf{x})}) + \frac{b}{k} \tag{2.32}$$

Beeing b a parameter tuned according with the problem needs, knowing that if $b = ln(N)$:

$$\min_{i\in\mathcal{I}}(h_i(\mathbf{x})) \leq h(\mathbf{x}) \tag{2.33a}$$

$$C \supseteq \bigcap_{i\in\mathcal{I}} C_i \tag{2.33b}$$

Making the closed-loop system less constrained but not as safe.

Union of Sets \ OR \ Maximum

$$\begin{aligned} h(\mathbf{x}) &= \frac{1}{k} ln(\sum_{i\in\mathcal{I}} e^{k\ h_i(\mathbf{x})}) \\ &\approx \max_{i\in\mathcal{I}}(h_i(\mathbf{x})) \end{aligned} \tag{2.34}$$

Beeing $k \in \mathbb{R}_{>0}$ as smoothing parameter. The derivative of $h(\mathbf{x})$ is equal to 2.30a but:

$$\lambda_i(\mathbf{x}) = e^{k(h_i(\mathbf{x}) - h(\mathbf{x}))} \tag{2.35}$$

With also $\sum_{i\in\mathcal{I}} \lambda_i(\mathbf{x}) = 1$.

Due to an over-aproximation of h relative to the max function, given the safe set C defined based on 2.6 it verifies:

$$\max_{i\in\mathcal{I}}(h_i(\mathbf{x})) \leq h(\mathbf{x}) \quad \forall \mathbf{x} \in \mathbb{R}^n \tag{2.36a}$$

$$C \supseteq \bigcup_{i\in\mathcal{I}} C_i \tag{2.36b}$$

But, $\lim_{k\to\infty} h(\mathbf{x}) = \max_{i\in\mathcal{I}}(h_i(\mathbf{x}))$ and $\lim_{k\to\infty} C = \bigcup_{i\in\mathcal{I}} C_i$

Now showing a different point of view, safety is a critical factor, so in order to tight the safe set C and potentially be contained within $\bigcup_{i\in\mathcal{I}} C_i$:

$$h(\mathbf{x}) = \frac{1}{k} ln(\sum_{i\in\mathcal{I}} e^{k\ h_i(\mathbf{x})}) - \frac{b}{k} \tag{2.37}$$

Beeing b a tunable parameter. Taking $b = ln(N)$:

$$\max_{i\in\mathcal{I}}(h_i(\mathbf{x})) \geq h(\mathbf{x}) \tag{2.38a}$$

$$C \subseteq \bigcup_{i\in\mathcal{I}} C_i \tag{2.38b}$$

Therefore the higher b, the safer the system 2.13 but possibly too constrained by the new CBF $h(\mathbf{x})$ 2.37.

2.1.4.2 Second-Order System

Following the explanations given in the previous subsubsection 2.1.4.1, now applying for a second-order system 2.11, more precisely, for a Backstepping 2.1.2 situation.

Given a CLF $V_0(\mathbf{x})$: and multiple CBF $h0_i(\mathbf{x}) \quad \forall i \in \mathcal{I}$ combined into a CBF $h_0(\mathbf{x})$ via 2.32 or 2.37 (according it is an AND or OR operation), through the QP 2.14 (or its closed-form solution 2.18) is obtained a differentiable and locally Lipschitz continuos theoretical controller $k_0(\mathbf{x}) : \mathbb{R}^n \to \mathbb{R}$.

In order to the second-order system 2.12 dynamic converge to $k_0(\mathbf{x})$ imposed dynamic, given multiple CBF $h_i(\mathbf{x}) \quad \forall i \in \mathcal{I}$:

$$V_\xi(\mathbf{x},\xi) = \frac{1}{2}\|\xi - k_0(\mathbf{x})\|^2 \tag{2.39a}$$

$$V(\mathbf{x},\xi) = V_0(\mathbf{x}) + V_\xi(\mathbf{x},\xi) \tag{2.39b}$$

$$h_i(\mathbf{x},\xi) = h0_i(\mathbf{x}) - V_\xi(\mathbf{x},\xi) \quad \forall i \in \mathcal{I} \tag{2.39c}$$

The resultant CBF $h_i(\mathbf{x},\xi) \quad \forall i \in \mathcal{I}$, one more time can be merged via 2.32 or 2.37 (according it is an AND or OR operation) into a single CBF $h(\mathbf{x},\xi)$. However there is another way in this case to have the new single CBF $h(\mathbf{x},\xi)$, so, considering the second-order system 2.11 and $lo \in \{\max, \min\}$:

$$\begin{aligned} \underset{i\in\mathcal{I}}{lo}(h0_i(\mathbf{x}) - V_\xi(\mathbf{x},\xi)) &= \underset{i\in\mathcal{I}}{lo}(h0_i(\mathbf{x})) - V_\xi(\mathbf{x},\xi) \approx \\ &\approx h_0(\mathbf{x}) - V_\xi(\mathbf{x},\xi) = \\ &= h(\mathbf{x},\xi) \end{aligned} \tag{2.40}$$

$$\dot{h}(\mathbf{x},\xi) = \underbrace{L_f h_0(\mathbf{x}) + L_G h_0(\mathbf{x})\xi - L_f V_\xi(\mathbf{x},\xi)}_{L_f h(\mathbf{x},\xi)} + \underbrace{(-L_G V_\xi(\mathbf{x},\xi))}_{L_G h(\mathbf{x},\xi)}\mathbf{u} \tag{2.41}$$

Having CBF h, the second-order solution, a differentiable and locally Lipschitz continuos controller $k(\mathbf{x},\xi) : \mathbb{R}^n \times \mathbb{R}^p \to \mathbb{R}$ is obtained using the QP 2.16 (or its closed-form solution 2.18).

Notes:

- There are other possible variations of a second-order system that invalidates 2.41 like the unicycle model that will be shown next chapter.

2.2 Single Parameter Optmization

The optimization of the respective parameter is based on minimizing a cost function $g : \mathbb{R} \rightarrow \mathbb{R}$, which in the context of this work expected to be quasi-convex [8] along the optimization variable $x \in \mathbb{R}$, and to do that there are algorithms such as the highly used gradient descent 2.2.2 and binary search 2.2.3 method.

2.2.1 Gradient of the Cost Function - Discretization

In both algorithms is make use of $g(x)$ gradient in order to essentially indicate the direction the optimization variable x must take in order to minimize the cost. More than that, if $g(x)$ is a quasi-convex function to quantifie the proximity to a local optimal point.

If the $\nabla g(x)$ extact equation is not possible to obtain then:

$$\nabla g(x) = \lim_{dev \rightarrow 0} \frac{g(x + dev) - g(x)}{dev} \tag{2.42}$$

Is approximated by:

$$\nabla g(x, dev) = \frac{g(x + dev) - g(x)}{dev} \tag{2.43}$$

With $dev \in \mathbb{R}$ sufficiently close to 0.

If $g(x)$ has relevant noise could compromise the value obtained of $\nabla g(x)$, it could be needed increase the number of samples ($nS \in \mathbb{R}$) in the neighborhood of x separated by $devN$ between them, so digitally:

Algorithm 1: Discrete Gradient Function (DGF)

Data: $nS, devN, dev, g, x$
Result: $mean(grad_i)$
$i \leftarrow -1$;
while $i < nS - 1$ **do**
  $devN_i \leftarrow \frac{1}{2} \times (i + (i \; mod \; 2)) \times devN$;
  $grad_i \leftarrow \nabla g(x + (-1)^i \times devN_i, (-1)^i \times dev)$; /* Equation 2.43 */
  $i \leftarrow i + 1$;
end

Notes:

- It is possible to digitally obtain $\nabla g : \mathbb{R}^n \rightarrow \mathbb{R}^n$ by otaining each row r separately with $dev_r \in \{dev_1, ..., dev_n\}$. Beeing dev_r a column vector, just equal to the divisor dev in the respective row element ($dev = \|dev_r\|$).

2.2.2 Gradient Descent

This method is characterized by an iterative update of the optimization variable x, based on the current variable value and function $g(x)$ gradient responsible to indicate the evolution direction of x in order to minimize the cost:

[8] A quasi-convex function $f : \mathcal{S} \rightarrow \mathbb{R}$, beeing $\mathcal{S}$ a convex set, verifies $\forall x, y \in \mathcal{S}$ and $\lambda \in [0; 1]$, $f(\lambda x + (1 - \lambda)y) \leq max\{f(x), f(y)\}$

$$x_{k+1} = x_k + \underbrace{(-a\nabla g(x_k))}_{step_k} \tag{2.44}$$

where a is an arbitrary constant for the step intensity adjustment . Assuming $g(x)$ is a quasi-convex function, the closure to local minimum the smaller the gradient (or step) module.

However, this method has another variations including adding "momentum" where is had in account the previous iteration step applied:

$$x_{k+1} = x_k + b\underbrace{(x_k - x_{k-1})}_{step_{k-1}} - a\nabla g(x_k) \tag{2.45}$$

where b is an argument of this algorithm like a responsible to tune the previous step intensity.

According to [1] there are also another two methods. Refering to one of them, this expands on the idea of momentum called Nesterov's Accelerated Gradient, using a predictive step based on the previous one:

$$\begin{aligned} y_k &= x_k + b(x_k - x_{k-1}) \\ x_{k+1} &= y_k + (\underbrace{-a\nabla g(y_k)}_{predictive\ step}) \end{aligned} \tag{2.46}$$

where a now tunes the predictive step.

Finally the Triple-Momentum method which presents an output equation and the parameters applied on the previous step are distinct in all usages of it:

$$\begin{aligned} z_k &= x_k + d(x_k - x_{k-1}) \\ y_k &= x_k + c(x_k - x_{k-1}) \\ x_{k+1} &= x_k + b(x_k - x_{k-1}) - a\nabla g(y_k) \end{aligned} \tag{2.47}$$

where a, b, c, d are the respective steps intensity tuning parameter.

Notes:

- The method supports multiple optimization variables.

2.2.3 Binary Search

This method is composed by two phases, beeing the first one resposible to obtain two extreme points $(xI, xS) \in \mathbb{R} \times \mathbb{R}$ such that the Bolzano theorem [9] is verified for the continuos gradient funtion ($x^* = \{x \in]xI; xS[: \exists x\ \nabla g(x) = 0\}$). In order to achieve that, is used an variation of the gradient descent method 2.44, with a starting point x_0:

$$x_{k+1} = x_k + \underbrace{(-a\nabla g(x_0))}_{step} \tag{2.48}$$

Where the $step$ is always equal. That beeing said, the phase 1 algorithm:

[9] Bolzano's theorem says that given the continuos function $f : [a; b] \subset \mathbb{R} \to \mathbb{R}$ and $f(a)f(b) < 0$, then $\exists x \in]a; b[$ such that $f(x) = 0$

Algorithm 2: Binary Search Phase 1

```
Data: xI, a
Result: (xI, xS)
x_0 ← xI;
xS ← xI;
while ∇g(xI) × ∇g(xS) > 0 do
    xS ← xI − a∇g(x_0) ;                    /* Equation 2.48 */
    if ∇g(xI) × ∇g(xS) > 0 then
        xI ← xS;
    end
end
```

The second phase consists in iteratively taking the extreme points and do a weighted mean obtaining a new delimiter that replace one of the previous points while preserving the application of the Bolzano theorem for next iterations, narrowing the interval and approximating the points to a local minimum.

Algorithm 3: Binary Search Phase 2

```
Data: xI, xS, tI, w
Result: xMid
while i < tI AND Conditions* do
    xMid ← w × xI + (1 − w) × xS ;          /* Weighted Mean */
    if ∇g(xMid) > 0 then
        xS ← xMid;
    else
        xI ← xMid;
    end
end
```

2.3 Constraint Contour Point

Having the objective of global assympyotically stability of a system, the system converges a desired equilibrium point $\bar{\mathbf{x}}_F$ but kept outside a bounded unsafe set $\mathcal{B} \subset \mathbb{R}^n$. Depending on certain decision algorithms, such as 2.10 which uses CLF-CBF, by pursuing another equilibrium point $\bar{\mathbf{x}}_I$ first, the path taken by the system's states $\mathbf{x} \in \mathbb{R}^n$ until $\bar{\mathbf{x}}_F$ could be oriented in order to be a better fit for problem demands, specially to deal more efficiently with the implications of an unsafe set $\mathcal{B}$.

2.3.1 Constraint Fit

The program formulated in 2.3.3.2 responsible to obtain a new equilibrium point is characterized by its application in two-dimensional Euclidian Space safety constraints. Since the safe set $\bar{\mathcal{B}}$ (complement of $\mathcal{B}$) corresponds to the 0-superlevel set of an unkown safety function $h_{unkown} : \mathbb{R}^n \to \mathbb{R}$, then via logical operations 2.1.4 smoth-approximations (AND 2.32, OR 2.37), combining the different 0-superlevel sets $C_i \subset \mathbb{R}^n \quad \forall i \in \mathcal{I} = \{1, ..., p\}$ of the the functions $h_i : \mathbb{R}^n \to \mathbb{R}$:

$$\begin{aligned} &h_{unkown}(\mathbf{x}) \approx h(\mathbf{x}) = merge(h_i(\mathbf{x})) \quad \forall i \in \mathcal{I} \\ &\bar{\mathcal{B}} \subseteq C = \{\mathbf{x} \in \mathbb{R}^n : h(\mathbf{x}) \geq 0\} \end{aligned} \tag{2.49}$$

$\bar{\mathcal{B}}$ can also be approximated by an ellipsoide:

$$h_{unkown}(\mathbf{x}) \approx h(\mathbf{x}) = \frac{1}{2}((\mathbf{O}\mathbf{x} - p)^\top \mathbf{A}_{\mathbf{xis}}(\mathbf{O}\mathbf{x} - p) - 1) \tag{2.50}$$

$$st. \quad \bar{\mathcal{B}} \subseteq C \tag{2.51}$$

With $\mathbf{O} \in \mathbb{R}^{c \times n}$ as constrained states observer (c equal to the constrained states), $\mathbf{A}_{\mathbf{xis}} \in \mathbb{R}^{c \times c}_{>0}$ and $p \in \mathbb{R}^c$ susceptible to an optimization that tries to minimize the size of the set $C \cap \mathcal{B}$ or at least choosing them such that is garanteed 2.51. Even though the ellipsoidal approximation is less accurate as is not able to capture the shapes of other figures than ellipses, it shows sufficient for some problems, it provides less complexity and given its properties, suited for some strategies planning.

2.3.2 State Propagation Simulation

2.3.2.1 Runge-Kutta (RK) Methods

Its a technique used to solve ordinarinary differential equation (ODE)s [10] initial-value problems [11] ($\frac{dy}{dx} = f(x, y)$ with (x_0, y_0)), i.e., to obtain an approximation of a non-linear system solution $y(x)$ at some point in its domain without using the higher order derivatives of it.

The **Euler** method shows the most degradation of local truncature error[12] as the required (arbitrary valued) step $\Delta x = x_{n+1} - x_n$, $n \in \mathbb{N}_0$ increases but quicker to compute.

[10] An ODE is characterized by having a independent variable $x \in \mathbb{R}$ (commonly time t) and dependent variables $y \in \mathbb{R}^n$ and its derivatives relative to x, such that an ODE corresponds to an equation equivalent to $g(x, y, \frac{dy}{dx}, ... \frac{d^n y}{dx^n}) = 0$

[11] A initial-value-problem is an ODE $\dot{\mathbf{y}}(t) = f(t, \mathbf{y}(t))$ with $f : \mathbb{R} \times \mathbb{R}^n \to \mathbb{R}^n$ with an initial condition t_0, y_0

[12] A local truncature error corresponds to the error caused by the integrative effect of an iteration of a Runge-Kutta method

$$y_{n+1} = y_n + \Delta x f(x_n, y_n) \tag{2.52}$$

This method has just one stage, while the next Runge-Kutta (RK)s expanded on it such that allow more. Those stages correspond to intermediate point derivatives based on predictive steps. Introducing to the **Explicitit Runge-Kutta** :

$$y_{n+1} = y_n + \Delta x \sum_{i=1}^{s} b_i k_i \tag{2.53a}$$

$$k_i = f(x_n + \Delta x c_i, y_n + \Delta x \sum_{j=1}^{i-1} a_{ij} k_j) \tag{2.53b}$$

Where s is the number of stages, $\mathbf{a}$ the Runge-Kutta (RK) matrix, with $a_{ij} \in \mathbb{R}$ and usually tuned such that $\sum_{j=1}^{i-1} a_{ij} = c_i$, $\mathbf{b} \in \mathbb{R}^s$ are the weights, so $\sum_{i=1}^{s} b_i = 1$, $\mathbf{c} \in \mathbb{R}^s$ are the nodes and $c_i \in [0;1]$. These parameters are usually presented by a Butcher tableau:

$$\begin{array}{c|cccc} 0 & & & & \\ c_2 & a_{21} & & & \\ \vdots & \vdots & \ddots & & \\ c_s & a_{s;1} & \dots & a_{s;s-1} & \\ \hline & b_1 & \dots & b_{s-1} & b_s \end{array} = \begin{array}{c|c} \mathbf{c} & \mathbf{a} \\ \hline & \mathbf{b}^\top \end{array} \tag{2.54}$$

The <u>RK 4th order</u> is defined by 2.53 and its tableau:

$$\begin{array}{c|cccc} 0 & & & & \\ \frac{1}{2} & \frac{1}{2} & & & \\ \frac{1}{2} & 0 & \frac{1}{2} & & \\ 1 & 0 & 0 & 1 & \\ \hline & \frac{1}{6} & \frac{1}{3} & \frac{1}{3} & \frac{1}{6} \end{array} \tag{2.55}$$

Although this method solution is slower comparatively to Euler 2.52, its accuracy is higher.

The **Adaptive Runge-Kutta** method, with a adaptive step size Δx according to the estimated local truncature error. If its lower than some treshold, the step size increases and the solutions obtained faster since that point, while if it increases is reduced promoting a accurate solution, but with $\Delta x \in [\Delta x_{min}; \Delta x_{max}]$. The error estimations are calculated based on two different methods of one order more than the other. For this new Runge-Kutta variation the Butcher Tableau is:

$$\begin{array}{c|c} \mathbf{c} & \mathbf{a} \\ \hline & \mathbf{b}^\top \\ & \mathbf{b}^{*\top} \end{array} \tag{2.56}$$

The low and high order solution are based equally on 2.53a:

$$y_{n+1} = y_n + \Delta x \sum_{i=1}^{s} b_i k_i \text{ (low-order/output sol.)} \tag{2.57a}$$

$$z_{n+1} = y_n + \Delta x \sum_{i=1}^{s} b_i^* k_i \text{ (high-order sol.)} \tag{2.57b}$$

Where the intermediate derivatives k_i functions keep the same equation 2.53b and are equal for both (low and hig order methods) reducing some computing time. The local truncature error estimations ig given by:

$$e_{n+1} = |z_{n+1} - y_{n+1}| \tag{2.58}$$

This error affects the new step value, the Dormand Prince method [2] with its particular update step size iteration. This method Butcher Tableau is:

$$\begin{array}{c|ccccccc}
0 & & & & & & & \\
\frac{1}{5} & \frac{1}{5} & & & & & & \\
\frac{3}{10} & \frac{3}{40} & \frac{9}{40} & & & & & \\
\frac{4}{5} & \frac{44}{45} & -\frac{56}{15} & \frac{32}{9} & & & & \\
\frac{8}{9} & \frac{19372}{6561} & -\frac{25360}{2187} & \frac{64448}{6561} & -\frac{212}{729} & & & \\
1 & \frac{9017}{3168} & -\frac{355}{33} & \frac{46732}{5247} & \frac{49}{176} & -\frac{5103}{18656} & & \\
1 & \frac{35}{384} & 0 & \frac{500}{1113} & \frac{125}{192} & -\frac{2187}{6784} & \frac{11}{84} & \\
\hline
 & \frac{35}{384} & 0 & \frac{500}{1113} & \frac{125}{192} & -\frac{2187}{6784} & \frac{11}{84} & 0 \\
 & \frac{5179}{57600} & 0 & \frac{7571}{16695} & \frac{393}{640} & -\frac{92097}{339200} & \frac{187}{2100} & \frac{1}{40}
\end{array} \tag{2.59}$$

The k_7 derivative is calculated at the predicted point obtained by the low order solution y_{n+1}. The update size step equation as:

$$\Delta x_{new} = \Delta x \left(\frac{\epsilon \Delta x}{2 e_{n+1}} \right)^{\frac{1}{5}} \tag{2.60}$$

With ϵ as the error tolerance equal to 10^{-5} in the default version of Matlab ode45.

2.3.2.2 State Propagation Simulation Algorithm

Finding a new equilibrium point $\bar{\mathbf{x}}_I$ requires simulate the state convergence. Aiming to simulate a system evolution based on a decision algorithm, given the ordinary diferential states dynamics equations $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$:

Algorithm 4: State Propagation Simulation (SPS)

Data: $f, \mathbf{x}_0, tI, \Delta t, arg^*$
Result: $\mathbf{x}$, $\mathbf{u}$
$i \leftarrow 0$;
while $i < tI$ ***AND*** *Conditions** **do**
  $\mathbf{u}_i \leftarrow decision^*(\mathbf{x}_i, arg^*)$; /* Decision Algorithm */
  $\mathbf{x}_{i+1} \leftarrow integration^*(f, \mathbf{x}_i, \mathbf{u}_i, \Delta t)$; /* Euler 2.52, RK 4 2.3.2.1 or DP 2.3.2.1... */
  $i \leftarrow i + 1$;
end

2.3.3 Constraint Contour Point Algorithm

2.3.3.1 Natural Convergence Direction

The new point is obtained accounting with the natural movement direction of those states when they would reach the unsafe zone, which is provided approximately by the propagation algorithm 4 if the safety constraint defined by h (2.49 or 2.50) was inactive.

Algorithm 5: Natural Convergence Direction Vector ($NCDV$)

Data: $f, \mathbf{x}_0, tI, \Delta t, args^*$
Result: dir

$(\mathbf{x}, \mathbf{u}) \leftarrow \text{SPS}(f, \mathbf{x}_0, tI, \Delta t, args^*)$ $\begin{bmatrix} conditions^* \leftarrow h(\mathbf{x}) > 0 \\ decision^* \\ integration^* \end{bmatrix}$; `/* Algorithm 4 */`

$dir \leftarrow \frac{f(\mathbf{x}_{END}, \mathbf{u}_{END})}{\|f(\mathbf{x}_{END}, \mathbf{u}_{END})\|}$; `/* System Dynamic at` $(\mathbf{x}_{END}, \mathbf{u}_{END})$ `Normalized */`

This vector points to the direction where the system can aproximate more of $\bar{\mathbf{x}}_F$ given some decision algorithm and assuming is always safe.

2.3.3.2 Constraint Tangent Point

From the vector $\mathbf{v} = \left[\begin{smallmatrix} v_1 \\ v_2 \end{smallmatrix}\right]$ obtained in 5 is deduced a tangent point in the respective constraint. This point is the most far from the eminent of colliding unsafe set C given a perpendicular direction to the obtained vector.Assuming it's always used the ellipsoidal fit 2.50, where $\mathbf{A_{xis}} = \left[\begin{smallmatrix} a_x & a_u \\ a_u & a_y \end{smallmatrix}\right] \in \mathbb{R}^{c \times c}_{>0}$, $\mathbf{O} \in \mathbb{R}^{c \times n}$ and $\mathbf{p} \in \mathbb{R}^c$, the new point could be obtained using a closed-form solution (to see the full deduction A.2):

$$d = \frac{a_u \sqrt{2}}{\sqrt{a_x a_y}} \tag{2.61}$$

$$l = \sqrt{\frac{a_x}{2a_y}} \tag{2.62}$$

$$q = \pm \sqrt{\frac{1}{\frac{a_x}{2}(v_1^2 + v_2^2) + a_y v_1^2}} \tag{2.63}$$

$$\mathbf{z}_i = \begin{bmatrix} v_2 q \\ v_1 q \end{bmatrix} \quad i \in \{1, 2, 3, 4\} \tag{2.64}$$

In order to obtain both solutions from the perpedincular vectors to $\mathbf{v}$:

$$\mathbf{r}_1 = \underset{\mathbf{z}_i \in \mathbb{R}^2}{\arg\min} = \begin{bmatrix} a_y v_2 & -a_x v_1 \end{bmatrix} \mathbf{z}_i \tag{2.65a}$$

$$\mathbf{r}_2 = -\mathbf{r}_1 \tag{2.65b}$$

With $\mathbf{r} = \begin{bmatrix} \mathbf{r}_1 & \mathbf{r}_2 \end{bmatrix}$. The respective points are obtained with:

$$\mathbf{s}_j = \frac{\begin{bmatrix} d\, r_{1j} + l\, r_{2j} \\ d\, r_{2j} - l\, r_{1j} \end{bmatrix}}{d^2 + l^2} + \mathbf{p} \quad j \in \{1, 2\} \tag{2.66}$$

The closest-point to the current state is chosed as the intermediate equilibrium point:

$$\mathbf{O}\bar{\mathbf{x}}_I = \underset{\mathbf{s}_j \in \mathbb{R}^2}{\arg\min} \|\mathbf{s}_j - \mathbf{x}\| \tag{2.67}$$

2.4 Proposed Decision Algorithms

All the proposed algorithms are based on CLF-CBF 2.1 decision algorithm, having the objective of stabilizing a system while prioritizing its safety. Using CLF-CBF allows a relative lower computational effort, in particular its closed form solution 2.1.3, compared to other techniques, however it lacks adaptive power as its constant parameters consistently dictates an inneficient system behavior for probably the majority of the encoutered scenarios.

2.4.1 Just Optimized (A-JO)

Optimizing parameters can improve the system behaviour for the scenario is deparing with. And an eventual optimization make its behaviour more suited to actuate in an multi-scenario environment.

Based on single parameter optimization 2.2, the proposed technique is centered on the slope α of the CBF constraint function $\alpha : \mathbb{R} \to \mathbb{R}$ (CLF-CBF QP 2.10) as an optimization variable:

$$\alpha(h) = \alpha h \tag{2.68}$$

The parameter is optimized using 2.44 but for that is required a cost function $g_{\mathbf{x}_0} : \mathbb{R} \to \mathbb{R}$ which accounts with a whole predction horizon:

$$\begin{aligned} g_{\mathbf{x}_0}(\alpha) &= g(\alpha, \mathbf{x}_0 = cte) \\ &= \textstyle\sum_{k=0}^{N} \mathbf{x}_{k+1}(\mathbf{x}_k, \mathbf{u}_k)^\top \mathbf{Q} \mathbf{x}_{k+1}(\mathbf{x}_k, \mathbf{u}_k) + \mathbf{u}_k(\alpha, \mathbf{x}_k)^\top \mathbf{R} \mathbf{u}_k(\alpha, \mathbf{x}_k) \\ &= \textstyle\sum_{k=0}^{N} \mathbf{x}_{k+1}(\alpha, \mathbf{x}_0)^\top \mathbf{Q} \mathbf{x}_{k+1}(\alpha, \mathbf{x}_0) + \mathbf{u}_k(\alpha, \mathbf{x}_0)^\top \mathbf{R} \mathbf{u}_k(\alpha, \mathbf{x}_0) \\ &= \mathbf{X}(\alpha, \mathbf{x}_0)^\top \tilde{\mathbf{Q}} \mathbf{X}(\alpha, \mathbf{x}_0) + \mathbf{U}(\alpha, \mathbf{x}_0)^\top \tilde{\mathbf{R}} \mathbf{U}(\alpha, \mathbf{x}_0) \end{aligned} \tag{2.69}$$

Where N is equivalent to the total of samples, $\mathbf{x}_0$ the present state, $\mathbf{Q} \in \mathbb{R}^n_{>0}$ is the states cost weight matrix, $\mathbf{R} \in \mathbb{R}^m_{>0}$ is the inputs cost weight matrix.

Because of the conditioned control input values given by CLF-CBF controller, the gradient $\nabla g_{\mathbf{x}_0}(\alpha)$ exact equation is difficult to have it, hence is required a discretization of it 2.2.1. Which leds to the exact function of $g_{\mathbf{x}_0}(\alpha)$ which is not known as $\mathbf{u}_k$ and $\mathbf{x}_{k+1}$ could not be obtained directly. Like it was said, $\mathbf{u}_k$ is obtained based on a closed-form CLF-CBF controller 2.1.3, governed by a CLF V and a CBF h. Meanwhile, $\mathbf{x}_{k+1}$ is given by an explicit RK2.3.2.1 in particular defined by the fourth order RK Butcher Tableau2.55, so given the system dynamics equation $f : \mathbb{R}^n \times \mathbb{R}^m \to \mathbb{R}^n$ and integration time interval Δt:

$$\begin{bmatrix} \mathbf{X}(\alpha, \mathbf{x}_0) & \mathbf{U}(\alpha, \mathbf{x}_0) \end{bmatrix} = SPS(f, \mathbf{x}_0, N, \Delta t, V, h, \alpha) \begin{bmatrix} conditions^* \leftarrow V(\mathbf{x}_i) \geq tol \\ decision^* \leftarrow \text{CLF-CBF } 2.1.3 \\ integration^* \leftarrow \text{RK4 } 2.55 \end{bmatrix} \tag{2.70}$$

With SPS (State Propagation Simulation) as the algorithm 4. So, choosing nS number of samples, $devS$ referent to the distance between them, dev the discretization step and with $g_{\mathbf{x}_0}(\alpha)$ given by 2.70:

$$\nabla g_{\mathbf{x}_0}(\alpha) = DGF(nS, devS, dev, g_{\mathbf{x}_0}, \alpha) \tag{2.71}$$

With DGF (Discrete Gradient Function) as the algorithm 1 and therefore enabling 2.2 and the optimization of the slope α.

Theoretically the CBF should garantee safety, however due to not be a continuos system (or integration errors), having a high α can lead to unsafe sets and can also provide a demand on a not possible control input values to keep the system safe, but if that happens is expected to reflect on the path taken, control inputs and therefore penalize in the cost function.

2.4.2 Double CLF

The CLF-CBF returns control input values based on purely the current state, resulting in global (infinite horizon) suboptimal solutions that could lead to states relatively close to the barrier (given some safety tunable defenitions) and unwanted inputs to deviate from it. So, changing the notion of stability by making the system follow another equilibrium point first could help mitigate the problems. This technique focus on the transition between two CLF aiming to converge for each of the equilibrium points, the current and next.

With constraint contour point algorithm 2.3.3 is obtained the new equilibrium point $\bar{\mathbf{x}_C} \in \mathbb{R}^n$. Given a system, with dynamics $f : \mathbb{R}^n \times \mathbb{R}^m \to \mathbb{R}^n$, at the sate $\mathbf{x}_0 \in \mathbb{R}^n$, a CLF V that targets $\bar{\mathbf{x}_N} \in \mathbb{R}^n$ and CBF h, the new equilibrium point could be obtained:

$$\mathbf{v} = NCDV(f, \mathbf{x}_0, N, \Delta t, V, h) \begin{bmatrix} decision^* \leftarrow \text{CLF-CBF } 2.1.3 \\ integration^* \leftarrow \text{RK4 } 2.55 \end{bmatrix} \tag{2.72}$$

Where $NCDV$ (Natural Convergence Direction Vector) is algorithm 5. Given the direction vector $\mathbf{v}$, assuming an ellipsoidal fit 2.50, with the constraint tangent point closed form solution 2.3.3.2 like this obtain $\bar{\mathbf{x}_C}$.

2.4.2.1 CLFs Summed (A-CLF-S)

From both equilibrium points is defined two CLF, V_C following the current equilibrium point $\bar{\mathbf{x}_C}$ and V_N stabilizing in $\bar{\mathbf{x}_N}$. By doing a wheighted mean between both CLF is defined a new one (a generalization for multiple CLF can be found in A.3):

$$V_1(\mathbf{x}) = \begin{cases} \sigma(\mathbf{x})V_C(\mathbf{x}) + (1 - \sigma(\mathbf{x}))V_N(\mathbf{x}) \\ \sigma(\mathbf{x}) = \frac{1}{1+e^{-\zeta V_C(\mathbf{x})}} \end{cases} \tag{2.73}$$

$$V(\mathbf{x}) = \begin{cases} V_1(\mathbf{x}) \quad \mathbf{x} \in \mathcal{S} \\ V_2(\mathbf{x}) = V_N(\mathbf{x}) \quad \mathbf{x} \in \bar{\mathcal{S}} \end{cases} \tag{2.74}$$

$$\mathcal{S} = \{\mathbf{x} \in \mathbb{R}^n : V_C(\mathbf{x}) \geq tol \cap \dot{V}_C(\mathbf{x}, \mathbf{u}) \leq 0 \cap \delta \leq tol_2\} \tag{2.75}$$

Where $\sigma(V_C(\mathbf{x}))$ is a sigmoide function responsible to the transition between the CLF and $\zeta \in \mathbb{R}_{\geq 0}$ is its slope. $\mathcal{S}$ refers to the just one time activation set of the respective CLF V_1, with δ referent to the CLF-CBF QP optimized relaxation variable.

The sets of possible control inputs, given the new CLF V_1 and assuming a high slope ζ ($\dot{\sigma}(\mathbf{x}, \mathbf{u}) \approx 0$), is approximately:

$$K_{V_1}(\mathbf{x}) = \Big\{\mathbf{u} \in \mathbb{R}^m : \dot{V}_1(\mathbf{x}, \mathbf{u}) = \dot{V}_C(\mathbf{x}, \mathbf{u})\sigma(\mathbf{x}) + (1 - \sigma(\mathbf{x}))\dot{V}_N(\mathbf{x}, \mathbf{u}) \\ \leq -\gamma(\mu\sigma(\mathbf{x})V_C(\mathbf{x}) + (1 - \sigma(\mathbf{x}))V_N(\mathbf{x}))\Big\} \tag{2.76}$$

With μ as a parameter with default value as $\frac{V_N(\mathbf{x})}{V_C(\mathbf{x})}$. The set $K_{V_2}(\mathbf{x})$ is equivalent to 2.5.

Finally, with A-JO 2.4.1 adapt the Control Barrier Function (CBF) slope α to the new Control Lyapunov Function (CLF) V and improve possibly even more the state convergence to the final equilibrium point.

2.4.2.2 CLFs Independent (A-CLF-I)

From both equilibrium points is defined two CLF, V_C following the current equilibrium point $\bar{\mathbf{x}_C}$ and V_N stabilizing in $\bar{\mathbf{x}_N}$. If $\zeta \to \infty$, according to equations 2.73 and 2.74:

$$V(\mathbf{x}) = \begin{cases} V_C(\mathbf{x}) & \mathbf{x} \in \mathcal{S} \\ V_N(\mathbf{x}) = V_N(\mathbf{x}) & \mathbf{x} \in \bar{\mathcal{S}} \end{cases} \tag{2.77}$$

$$\mathcal{S} = \{\mathbf{x} \in \mathbb{R}^n : V_C(\mathbf{x}) \geq tol \cap \dot{V}_C(\mathbf{x}, \mathbf{u}) \leq 0 \cap \delta \leq tol_2\} \tag{2.78}$$

And if $\mu = \frac{V_N(\mathbf{x})}{V_C(\mathbf{x})}$ then:

$$K_{V_C}(\mathbf{x}) = \Big\{\mathbf{u} \in \mathbb{R}^m : \dot{V}_C(\mathbf{x}, \mathbf{u}) \leq -\gamma(V_N(\mathbf{x}))\Big\} \tag{2.79}$$

The set $K_{V_N}(\mathbf{x})$ is is defined by 2.5.

Like in the previous technique 2.4.2.1, it is possible to apply A-JO 2.4.1 and improve the trajectory.

2.4.2.3 CLFs Unified (A-CLF-U)

Estou a pensar em deixar de parte para já, e como os resultados desta tecnica não são grande coisa acho que vou me focar no capitulo 3 para já.

3

Results

The algorithms seen in 2.4, namely, A-JO, A-CLF-S and A-CLF-I, are going to be implemented in different systems, in a unicycle 3.1.2 and orbital system 3.1.3, and subject to different types of experiments according to the system in question (unicycle 3.1.2.3 or orbital 3.1.3.3). In each of the contexts, there is an analisys of the techniques and a comparation between them highlingting some relevant data.

3.1 Setup

3.1.1 Decision Algorithms

3.1.1.1 A-JO

The parameter α is optimized based on a cost function $g_{\mathbf{x}_0}(\alpha)$ 2.69, which takes $N = 6000$ samples and has the weight cost matrices $\mathbf{Q} = \mathbf{I}_{2\times2}$ and $\mathbf{R} = 5 \cdot \mathbf{I}_{2\times2}$.

The function SPS 2.70 and DGF 2.71 parameters used in these experiments are given by:

Table 3.1: SPS 2.70 Parameters

f	$\mathbf{x}_0$	N	Δt	$V,\ h,\ \alpha_{ini}$
3.1 or **??**	$[0\ 0]^\top$	6000	0.005	3.1.2.2 or **??**

Table 3.2: DGF 2.71 Parameters

nS	$devS$	dev	$g_{\mathbf{x}_0}(\alpha)$	α_{ini}
3	0.01	0.005	2.69	3.1.2.2 or **??**

3.1.1.2 Double CLF

The double CLF algorithms 2.4.2 functions parameters are defined by:

Table 3.3: NCDV 2.72 Parameters

f	$\mathbf{x}_0$	N	Δt	V, h
3.1 or **??**	$[0\ 0]^\top$	6000	0.005	3.1.2.2 or **??**

Table 3.4: Double CLF 2.4.2 Transition Parameters

Algorithms	η	μ	tol	tol_2
A-CLF-S 2.4.2.1	1	$\frac{V_N(\mathbf{x})}{V_C(\mathbf{x})}$	$0.03 \cdot V_C(\mathbf{x})$	999
A-CLF-I 2.4.2.2	—	—	$0.05 \cdot V_C(\mathbf{x})$	50

3.1.2 Unicycle

This example consists in drive a unicycle to a specific location **ref** $\in \mathbb{R}^2$. Drive a vehicle implies the possibilitie of founding some obstacles, and while a collision could mean a fatality there are other factors as the fuel/energy consumed and how fast to achieve the wanted location, that beeing said, this represents an example of safety-critical control subject to an optimization of its path.

3.1.2.1 Dynamics

This unicycle doesn't loose traction in its wheels, and it is not constrained by its control inputs or its consume. It is also not affected by any perturbation such as drag force characteristic from the enviroment. It can be modeled mathematically as a relative two degree system:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \frac{1}{2}cos\theta & \frac{1}{2}cos\theta \\ \frac{1}{2}sin\theta & \frac{1}{2}sin\theta \\ -L & L \end{bmatrix} \begin{bmatrix} v_l \\ v_r \end{bmatrix} \tag{3.1}$$

Where the states are the (x, y) 2D position and θ the orientation, and the control inputs $\mathbf{u} \in \mathbb{R}^2$ are the left and right wheel speed v_l and v_r respectively, in addition, the constant $L = 10m^{-1}$ refers to the inverse of the distance between the two wheels.

3.1.2.2 CLF-CBF Formulation

As the decision algoritms subject to comparations 3.1.1 are governed by the CLF-CBF 2.1 backstepping technique 2.1.2, the system can be written as:

$$\begin{aligned} \dot{\mathbf{x}} &= f_0(\mathbf{x}) + g_0(\mathbf{x}, \xi)\mathbf{u} \\ \dot{\xi} &= f_1(\mathbf{x}, \xi) + g_1(\mathbf{x}, \xi)\mathbf{u} \end{aligned} \tag{3.2}$$

With $\mathbf{x} = \left[\begin{smallmatrix} x \\ y \end{smallmatrix}\right]$, $\xi = \left[\begin{smallmatrix} cos\theta \\ sin\theta \end{smallmatrix}\right]$, $f_0(\mathbf{x}) = \mathbf{0}_{2\times1}$, $g_0(\mathbf{x}, \xi) = \left[\begin{smallmatrix} \frac{1}{2}\xi_1 & \frac{1}{2}\xi_1 \\ \frac{1}{2}\xi_2 & \frac{1}{2}\xi_2 \end{smallmatrix}\right]$, $f_1(\mathbf{x}, \xi) = \mathbf{0}_{2\times1}$ and $g_1(\mathbf{x}, \xi) = \left[\begin{smallmatrix} \xi_2 L & -\xi_2^L \\ -\xi_1 L & \xi_2 L \end{smallmatrix}\right]$.

Since it's a second-order system, to use backstepping and obtain the first-order solution $\mathbf{k}_0(\mathbf{x})$ it is considered the system:

$$\dot{\mathbf{x}} = k_0(\mathbf{x}) \tag{3.3}$$

In order to make the vehicle to the wanted position is formulated a CLF V_0 which actively works to stabilize the system in the constant **ref**.

$$V_0(\mathbf{x}) = \frac{1}{2}(\mathbf{x} - \mathbf{ref})^\top(\mathbf{x} - \mathbf{ref}) \tag{3.4a}$$

$$\dot{V}_0(\mathbf{x}) = \underbrace{(\mathbf{x} - \mathbf{ref})^\top}_{L_G V_0(\mathbf{x})} \mathbf{k}_0(\mathbf{x}) \tag{3.4b}$$

$$\gamma_0(\mathbf{x}) = V_0(\mathbf{x}) \tag{3.4c}$$

The safety is defined according to the CBF h_0 which restricts the obstacles using a ellipsoidal fit 2.50:

$$h_0(\mathbf{x}) = (\mathbf{x} - \mathbf{p})^\top \mathbf{A_{xis}}(\mathbf{x} - \mathbf{p}) - 1 \tag{3.5a}$$

$$\dot{h}_0(\mathbf{x}) = \underbrace{((\mathbf{A_{xis}}(\mathbf{x} - \mathbf{p}))^\top + (\mathbf{x} - \mathbf{p})^\top \mathbf{A_{xis}})}_{L_G h_0(\mathbf{x})}\mathbf{k}_0(\mathbf{x}) \tag{3.5b}$$

$$\alpha_0(\mathbf{x}, \alpha) = \alpha h_0(\mathbf{x}) \tag{3.5c}$$

With $\mathbf{A_{xis}} \in \mathbb{R}^{2\times 2}_{\geq 0}$ referent to the ellipse axis and $\mathbf{p} \in \mathbb{R}^2$ equal to the center of the ellipse, both stipulated in 3.1.2.3. Given the system 3.2, the solution $\mathbf{k}_0(\mathbf{x})$, the CLF V and CBF V are defined (according to 2.15) as:

$$V(\mathbf{x}, \xi) = V_0(\mathbf{x}) + 1 - cos(\theta - \theta_0(\mathbf{x})) \tag{3.6a}$$

$$\dot{V}(\mathbf{x}, \xi) = \underbrace{\left(((\mathbf{x} - \mathbf{ref}))^\top g_0(\mathbf{x}, \mathbf{k}_{0,\xi}) + sin(\theta - \theta_0(\mathbf{x}))\begin{bmatrix} -L & L \end{bmatrix}\right)}_{L_G V(\mathbf{x},\xi)}\mathbf{k}(\mathbf{x}) \tag{3.6b}$$

$$\gamma(\mathbf{x}) = V(\mathbf{x}) \tag{3.6c}$$

$$h(\mathbf{x}, \xi) = h_0(\mathbf{x}) + 1 - cos(\theta - \theta_0(\mathbf{x})) \tag{3.7a}$$

$$\dot{h}(\mathbf{x}, \xi) = \underbrace{\left(((\mathbf{A_{xis}}(\mathbf{x} - \mathbf{p}))^\top + (\mathbf{x} - \mathbf{p})^\top \mathbf{A_{xis}})g_0(\mathbf{x}, \mathbf{k}_{0,\xi}) + sin(\theta - \theta_0(\mathbf{x}))\begin{bmatrix} -L & L \end{bmatrix}\right)}_{L_G h(\mathbf{x},\xi)}\mathbf{k}(\mathbf{x}) \tag{3.7b}$$

$$\alpha(\mathbf{x}, \xi, \alpha = 1.2) = \alpha h(\mathbf{x}, \xi) \tag{3.7c}$$

With $\mathbf{k}_{0,\xi}(\mathbf{x})$ and $\theta(\mathbf{x})$ given by:

$$\mathbf{k}_{0,\xi}(\mathbf{x}) = \frac{\mathbf{k}_0(\mathbf{x})}{||\mathbf{k}_0(\mathbf{x})||} \tag{3.8}$$

$$\theta_0(\mathbf{x}) = angle(\mathbf{k}_{0,\xi,1}(\mathbf{x}) + \mathbf{i}\,\mathbf{k}_{0,\xi,2}(\mathbf{x})) \tag{3.9}$$

3.1.2.3 Type of Experiments

-falar do tipo de simulaçoes que se vai fazer
-falar dos valores tomados pelas axis das elipses

3.1.3 Orbital

3.1.3.1 Dynamics

-apresentar modelo orbital
-CLF e CBF deste modelo e referindo o facto do fit ser elipsoidal

3.1.3.2 CLF-CBF Formulation

3.1.3.3 Type of Experiments

-falar do tipo de simulaçoes que se vai fazer
-falar dos valores tomados pelas axis das elipses

3.1.4 Some Data

The techniques will be evaluated having in account a cost, given by the function $g(\mathbf{X}, \mathbf{U})$, equivalent to $g_{\bar{\mathbf{x}}_0}(\alpha)$ defined before 3.1.1.1, such as the weight cost matrices, $\mathbf{Q} = \mathbf{I}_{2\times2}$ and $\mathbf{R} = 5 \cdot \mathbf{I}_{2\times2}$, besides that the function is calculated with $N = 6000$ samples. It is also presented the time the system f takes to reach the equilibrium point.

The computational time for each experiment is given based on a single test (one sample Monte Carlo) using the the processor *Intel(R) Core(TM) i7-8650U CPU @ 1.90GHz*.

3.2 Experiments

The dynamical system propagation with respective decision algorithm 3.1.1 during the experiments is simulated using the Dormand Prince integrative method 2.59. So, given a dynamical system (unicycle 3.1 or orbital **??**) $f : \mathbb{R}^n \to \mathbb{R}^n$, at $\mathbf{x}_0 \in \mathbb{R}^n$, a CLF V and a CBF h, a sampling time $\Delta t = 0.005$ seconds and a total of 6000 iterations totaling a simulation of 30 seconds:

$$\begin{bmatrix} \mathbf{X} & \mathbf{U} \end{bmatrix} = SPS(f, \mathbf{x}_0, 6000, 0.005, V, h) \begin{bmatrix} conditions* \leftarrow V(\mathbf{x}) \geq tol \\ decision^* \leftarrow \text{algorithms } 3.1.1 \\ integration^* \leftarrow \text{DP } 2.59 \end{bmatrix} \tag{3.10}$$

Where SPS (State Propagation Simulation) is the algorithm 4.

-mostrar as simulaçoes
-explicar os resultados obtidos

4

Conclusion

Bibliography

[1] H. Hao, D. Silvestre, and C. Silvestre. "Analysis of gradient descent algorithms: Discrete to continuous domains and circuit equivalents". In: *Systems & Control Letters* 203 (2025), p. 106146 (cit. on p. 13).

[2] T. Kimura. "On dormand-prince method". In: *Jpn. Malaysia Tech. Instit* 40 (2009), pp. 1–9 (cit. on p. 17).

[3] H. Matias and D. Silvestre. "Hybrid Lyapunov and Barrier Function-Based Control with Stabilization Guarantees". In: *arXiv preprint arXiv:2504.09760* (2025) (cit. on p. 7).

[4] T. G. Molnar and A. D. Ames. "Composing control barrier functions for complex safety specifications". In: *IEEE Control Systems Letters* 7 (2023), pp. 3615–3620 (cit. on p. 8).

A

Deductions

A.1 Closed-Form QP given CLF-CBF Constraints

Following the QP 2.17, suppose there is only one CLF, given the differentiable quadratic objective function $f : \mathbb{R}^{m+1} \to \mathbb{R}$:

$$f(\mathbf{u}, \delta) = \frac{1}{2}(\mathbf{u}^\top R\mathbf{u} + p\delta^2) \tag{A.1}$$

where $\mathbf{u} \in \mathbb{R}^m$, $\delta \in \mathbb{R}$, $R \in \mathbb{R}^{m\times m}$ and $p \in \mathbb{R}$.

And the optimization problem constrained just by a CLF V and a CBF h presented in the respective differentiable functions $g_i(\mathbf{x}, \mathbf{u}, \delta) : \mathbb{R}^{n+m+1} \to \mathbb{R}, \quad \forall i \in \{1, 2\}$:

$$\begin{aligned} &\underbrace{F_V(\mathbf{x}) + L_G V(\mathbf{x})\mathbf{u} - \delta}_{g_1(\mathbf{x},\mathbf{u},\delta)} \leq 0 \\ &\underbrace{F_h(\mathbf{x}) + L_G h(\mathbf{x})\mathbf{u}}_{-g_2(\mathbf{x},\mathbf{u},\delta)} \geq 0 \end{aligned} \tag{A.2}$$

whith $\mathbf{x} \in \mathbb{R}^n$ (corresponding not to an optimzation variable but as a constant, as a current state of the system **??**).

The constrained optimization problem can form the Lagrangian function:

$$\mathcal{L}(\mathbf{x}, \mathbf{u}, \delta, \lambda) = f(\mathbf{u}, \delta) + \lambda^\top g(\mathbf{x}, \mathbf{u}, \delta) \tag{A.3}$$

where $\lambda \in \mathbb{R}^2$ is called the KKT multiplier.

Aiming to hold the stationarity condition and find the optimal (minimal) solution:

$$\nabla(\mathcal{L}(\mathbf{x}, \mathbf{u}, \delta, \lambda)) = 0 \tag{A.4}$$

Supposing $g(\mathbf{x}, \mathbf{u}, \delta) = \left[\begin{smallmatrix} g_1(\mathbf{x},\mathbf{u},\delta) \\ g_2(\mathbf{x},\mathbf{u},\delta) \end{smallmatrix}\right] = F(\mathbf{x}) + L_G(\mathbf{x})\mathbf{u} + \left[\begin{smallmatrix} -\delta \\ 0 \end{smallmatrix}\right]$ with $L_G(\mathbf{x}) = \left[\begin{smallmatrix} L_G V(\mathbf{x}) \\ -L_G h(\mathbf{x}) \end{smallmatrix}\right]$ and $F(\mathbf{x}) = \left[\begin{smallmatrix} F_V(\mathbf{x}) \\ -F_h(\mathbf{x}) \end{smallmatrix}\right]$:

$$\begin{aligned} \frac{\partial \mathcal{L}(\mathbf{x}, \mathbf{u}, \delta, \lambda)}{\partial \mathbf{u}} = 0 &\iff \mathbf{u}^\top R + \lambda^\top L_G = 0 \\ &\iff R^\top \mathbf{u} + L_G^\top \lambda = 0 \\ &\iff \mathbf{u} = -(R^\top)^{-1} L_G^\top \lambda = -L_G^\top \lambda \quad \text{(Considering } R = \mathbf{I}_{m\times m} \\ &\qquad\qquad\qquad\qquad\qquad\qquad \text{i.e, assuming input consume equally)} \end{aligned} \tag{A.5}$$

$$\frac{\partial \mathcal{L}(\mathbf{x}, \mathbf{u}, \delta, \lambda)}{\partial \delta} = 0 \iff p\delta + \lambda^\top \begin{bmatrix} -1 \\ 0 \end{bmatrix} = 0$$
$$\iff \delta = p^{-1} \begin{bmatrix} 1 & 0 \end{bmatrix} \lambda \tag{A.6}$$

$$\begin{aligned}
\frac{\partial \mathcal{L}(\mathbf{x}, \mathbf{u}, \delta, \lambda)}{\partial \lambda} = 0 &\iff F(\mathbf{x}) + L_G(\mathbf{x})\mathbf{u} + \begin{bmatrix} -\delta \\ 0 \end{bmatrix} = 0 \\
&\iff F(\mathbf{x}) - \begin{bmatrix} \delta \\ 0 \end{bmatrix} - L_G(\mathbf{x})L_G^\top(\mathbf{x})\lambda = 0 \\
&\iff F(\mathbf{x}) - \begin{bmatrix} p^{-1} \begin{bmatrix} 1 & 0 \end{bmatrix} \lambda \\ 0 \end{bmatrix} - L_G(\mathbf{x})L_G^\top(\mathbf{x})\lambda = 0 \\
&\iff F(\mathbf{x}) - \left(\begin{bmatrix} p^{-1} & 0 \\ 0 & 0 \end{bmatrix} + L_G(\mathbf{x})L_G^\top(\mathbf{x}) \right) \lambda = 0 \\
&\iff \lambda = \left(\begin{bmatrix} p^{-1} & 0 \\ 0 & 0 \end{bmatrix} + L_G(\mathbf{x})L_G^\top(\mathbf{x}) \right)^{-1} F(\mathbf{x})
\end{aligned} \tag{A.7}$$

$$\left(\begin{bmatrix} p^{-1} & 0 \\ 0 & 0 \end{bmatrix} + L_G(\mathbf{x})L_G(\mathbf{x})^\top(\mathbf{x}) \right)^{-1} = \begin{bmatrix} p^{-1} + L_G V(\mathbf{x}) L_G V^\top(\mathbf{x}) & -L_G V(\mathbf{x}) L_G h^\top(\mathbf{x}) \\ -L_G h(\mathbf{x}) L_G V^\top(\mathbf{x}) & L_G h(\mathbf{x}) L_G h^\top(\mathbf{x}) \end{bmatrix}^{-1}$$

$$= \frac{adj(.)}{det(.)} = \frac{adj(.)}{\Delta} \tag{A.8}$$

$$adj(.) = \begin{bmatrix} L_G h(\mathbf{x}) L_G h^\top(\mathbf{x}) & L_G V(\mathbf{x}) L_G h^\top(\mathbf{x}) \\ L_G h(\mathbf{x}) L_G V^\top(\mathbf{x}) & p^{-1} + L_G V(\mathbf{x}) L_G V^\top(\mathbf{x}) \end{bmatrix} \tag{A.9}$$

$$\begin{aligned}
\Delta &= L_G h(\mathbf{x}) L_G V^\top(\mathbf{x}) L_G V(\mathbf{x}) L_G h^\top(\mathbf{x}) + (p^{-1} + L_G V(\mathbf{x}) L_G V^\top(\mathbf{x}))(L_G h(\mathbf{x}) L_G h^\top(\mathbf{x})) \\
&= (L_G h(\mathbf{x}) L_G V^\top(\mathbf{x}))^2 + (p^{-1} + \|L_G V(\mathbf{x})\|^2)\|L_G h(\mathbf{x})\|^2 \qquad \text{(Since there are just one CLF and CBF)}
\end{aligned} \tag{A.10}$$

$$\begin{aligned}
\lambda &= \Delta^{-1} adj(.) F(\mathbf{x}) \\
&= \Delta^{-1} \begin{bmatrix} -L_G V(\mathbf{x}) L_G h(\mathbf{x})^\top F_h + \|L_G h(\mathbf{x})\|^2 F_V \\ \\ L_G h(\mathbf{x}) L_G V(\mathbf{x})^\top F_V - (p^{-1} + \|L_G V(\mathbf{x})\|^2) F_h \end{bmatrix} \\
&= \begin{bmatrix} \lambda_1 \\ \lambda_2 \end{bmatrix}
\end{aligned} \tag{A.11}$$

$$\begin{aligned}
\mathbf{u} &= -L_G^\top \lambda \\
&= -L_G V(\mathbf{x}) \lambda_1 + L_G h(\mathbf{x}) \lambda_2
\end{aligned} \tag{A.12}$$

According with the dual feasibility condition the respective constraint is active if $\lambda_i \geq 0$, for $i = \{1, 2\}$, so given the activation subdomain $\mathcal{S}_j$, for $j = \{1, 2, 3, 4\}$:

If any is active then:

$$\mathcal{S}_4(\mathbf{x}) = \{(\mathbf{x}) \in \mathbb{R}^n : \lambda_1 < 0 \cap \lambda_2 < 0\} \tag{A.13}$$

$$k_4 = \mathbf{0} \tag{A.14}$$

If both, the CLF and CBF is active then:

$$\mathcal{S}_3(\mathbf{x}) = \{(\mathbf{x}) \in \mathbb{R}^n : \lambda_1 \geq 0 \cap \lambda_2 \geq 0\} \tag{A.15}$$

$$k_3 = -L_G V(\mathbf{x})\lambda_1 + L_G h(\mathbf{x})\lambda_2 \tag{A.16}$$

If just the CBF is active:

$$\mathcal{S}_2(\mathbf{x}) = \{(\mathbf{x}) \in \mathbb{R}^n : \lambda_1 \geq 0 \cap \lambda_2 < 0\} \tag{A.17}$$

Or just the CLF is active:

$$\mathcal{S}_1(\mathbf{x}) = \{(\mathbf{x}) \in \mathbb{R}^n : \lambda_1 < 0 \cap \lambda_2 \geq 0\} \tag{A.18}$$

The equation A.12 is not suitable and doesn't support the KKT conditions and therefore there is a need to obtain the controller $k_j(\mathbf{x}) : \mathbb{R}^n \to \mathbb{R}$, for $j = \{3, 4\}$, by removing from the optimization problem the respective inactive constraint.

Considering just the **CBF active**, the new Lagrangian function:

$$\mathcal{L}(\mathbf{x}, \mathbf{u}, \delta, \lambda_2) = f(\mathbf{u}, \delta) + \lambda_2^\top g_2(\mathbf{x}, \mathbf{u}, \delta) \tag{A.19}$$

One more time, keeping with the stationarity condition:

$$\nabla(\mathcal{L}(\mathbf{x}, \mathbf{u}, \delta, \lambda_2)) = 0 \tag{A.20}$$

$$\begin{aligned} \frac{\partial \mathcal{L}(\mathbf{x}, \mathbf{u}, \delta, \lambda_2)}{\partial \mathbf{u}} = 0 &\iff \mathbf{u}^\top R - \lambda_2^\top L_G h(\mathbf{x}) = 0 \\ &\iff R^\top \mathbf{u} - L_G h^\top(\mathbf{x})\lambda_2 = 0 \\ &\iff \mathbf{u} = (R^\top)^{-1} L_G h^\top(\mathbf{x})\lambda_2 = L_G h^\top \lambda_2 \quad (R = \mathbf{I}_{m\times m}) \end{aligned} \tag{A.21}$$

$$\begin{aligned} \frac{\partial \mathcal{L}(\mathbf{x}, \mathbf{u}, \delta, \lambda_2)}{\partial \delta} = 0 &\iff p\delta = 0 \\ &\iff \delta = 0 \end{aligned} \tag{A.22}$$

$$\begin{aligned}\frac{\partial \mathcal{L}(\mathbf{x},\mathbf{u},\delta,\lambda_2)}{\partial \lambda_2} = 0 &\iff -F_h(\mathbf{x}) - L_G h(\mathbf{x})\mathbf{u} = 0 \\ &\iff F_h(\mathbf{x}) + L_G h(\mathbf{x}) L_G h^\top(\mathbf{x})\lambda_2 = 0 \\ &\iff \lambda_2 = -(L_G h(\mathbf{x}) L_G h^\top(\mathbf{x}))^{-1} F_h(\mathbf{x}) \\ &\iff \lambda_2 = -\|L_G h(\mathbf{x})\|^{-2} F_h(\mathbf{x})\end{aligned} \tag{A.23}$$

Combinig the input equation A.21 and the KKT multiplier formula A.23:

$$\begin{aligned}k_2(\mathbf{x}) &= L_G h^\top(\mathbf{x})\lambda_2 \\ &= -L_G h^\top(\mathbf{x})\|L_G h(\mathbf{x})\|^{-2} F_h(\mathbf{x})\end{aligned} \tag{A.24}$$

Finally just with the **CLF active** :

$$\mathcal{L}(\mathbf{x},\mathbf{u},\delta,\lambda_1) = f(\mathbf{u},\delta) + \lambda_1^\top g_1(\mathbf{x},\mathbf{u},\delta) \tag{A.25}$$

According with the stationarity condition:

$$\nabla(\mathcal{L}(\mathbf{x},\mathbf{u},\delta,\lambda_1)) = 0 \tag{A.26}$$

$$\begin{aligned}\frac{\partial \mathcal{L}(\mathbf{x},\mathbf{u},\delta,\lambda_1)}{\partial \mathbf{u}} = 0 &\iff \mathbf{u}^\top R + \lambda_1^\top L_G h(\mathbf{x}) = 0 \\ &\iff \mathbf{u} = -L_G V^\top \lambda_1 \quad (R = \mathbf{I}_{m\times m})\end{aligned} \tag{A.27}$$

$$\begin{aligned}\frac{\partial \mathcal{L}(\mathbf{x},\mathbf{u},\delta,\lambda_1)}{\partial \delta} = 0 &\iff p\delta - \lambda_1 = 0 \\ &\iff \delta = \frac{\lambda_1}{p}\end{aligned} \tag{A.28}$$

$$\begin{aligned}\frac{\partial \mathcal{L}(\mathbf{x},\mathbf{u},\delta,\lambda_1)}{\partial \lambda_1} = 0 &\iff F_V(\mathbf{x}) + L_G V(\mathbf{x})\mathbf{u} - \delta = 0 \\ &\iff F_V(\mathbf{x}) - L_G V(\mathbf{x}) L_G h^\top(\mathbf{x})\lambda_1 - \delta = 0 \\ &\iff F_V(\mathbf{x}) - L_G V(\mathbf{x}) L_G h^\top(\mathbf{x})\lambda_1 - p^{-1}\lambda_1 = 0 \\ &\iff \lambda_1 = (\|L_G V(\mathbf{x})\|^2 + p^{-1})^{-1} F_V(\mathbf{x})\end{aligned} \tag{A.29}$$

Replacing into the input equation A.27 the KKT multiplier formula A.29:

$$\begin{aligned}k_1(\mathbf{x}) &= -L_G V^\top(\mathbf{x})\lambda_1 \\ &= -L_G V^\top(\mathbf{x})(\|L_G V(\mathbf{x})\|^2 + p^{-1})^{-1} F_V(\mathbf{x})\end{aligned} \tag{A.30}$$

Syntethizing like this the controller $k(\mathbf{x})$ that complies all the necessary conditions:

$$\mathbf{k}(\mathbf{x}) = \begin{cases} \mathbf{k}_1(\mathbf{x}), & (\mathbf{x}) \in \mathcal{S}_1 \\ \mathbf{k}_2(\mathbf{x}), & (\mathbf{x}) \in \mathcal{S}_2 \\ \mathbf{k}_3(\mathbf{x}), & (\mathbf{x}) \in \mathcal{S}_3 \\ \mathbf{0}_{m\times 1}\ , & (\mathbf{x}) \in \mathcal{S}_4 \end{cases} \tag{A.31}$$

A.2 Closed-Form Ellipsoide Tangent Point

A.3 Multiple CLF Transition Formulation

From each equilibrium points is defined a CLF, $V_{N_{j-1}}$ stabilizing in $\bar{\mathbf{x}}_{N_{j-1}}$, $j \in \{1, ..., K+1\}$, beeing j referent to the in the moment next (after the current) equilibrium state which is expected the system to follow and $K+1$ the total equilibrium points:

$$V_{C_{j|j}}(\mathbf{x}) = V_{N_{j-1}}(\mathbf{x}) \quad j \in \{1, ..., K+1\} \tag{A.32}$$

$$V_{C_{j|i+1}}(\mathbf{x}) = \begin{cases} \sigma_{j|i}(\mathbf{x}) V_{C_{j|i}}(\mathbf{x}) + (1 - \sigma_{j|i}(\mathbf{x})) V_{N_i}(\mathbf{x}) & i \in \{j, ..., K\} \\ \sigma_{j|i}(\mathbf{x}) = \frac{1}{1+e^{-\zeta V_{C_{j|i}}(\mathbf{x})}} \end{cases} \tag{A.33}$$

$$V(\mathbf{x}) = \begin{cases} V_j(\mathbf{x}) = V_{C_{j|K+1}}(\mathbf{x}) & \mathbf{x} \in \mathcal{S}_j \\ V_{K+1}(\mathbf{x}) = V_{N_K}(\mathbf{x}) & \mathbf{x} \in \bigcup_{j=0}^{K} \bar{\mathcal{S}}_j \end{cases} \tag{A.34}$$

$$\mathcal{S}_j = \{\mathbf{x} \in \mathbb{R}^n : V_{C_{j|j}}(\mathbf{x}) \geq tol \cap \dot{V}_{C_{j|j}}(\mathbf{x}, \mathbf{u}) \leq 0 \cap \delta \leq tol_2\} \tag{A.35}$$

Where $\sigma(V_{C_{j|i}}(\mathbf{x}))$ is a sigmoide function responsible to the transition between the CLF $V_{C_{j|i}}$ and V_{N_i}, $\zeta \in \mathbb{R}_{\geq 0}$ is its slope and $\mathcal{S}_j$ the just one time activation set of the respective CLF V_j, with δ referent to the CLF-CBF QP optimized relaxation variable.

The sets of possible control inputs, given the new CLF V_j and assuming a a high slope ζ ($\dot{\sigma}(\mathbf{x}, \mathbf{u}) \approx 0$), is approximately:

$$\begin{aligned} K_{V_j}(\mathbf{x}) = \Big\{ \mathbf{u} \in \mathbb{R}^m : \dot{V}_j(\mathbf{x}, \mathbf{u}) = & \Big(\dot{V}_{N_{j-1}}(\mathbf{x}, \mathbf{u}) \prod_{i=j}^{K} \sigma_{j|i}(\mathbf{x}) \Big) + \Big((1 - \sigma_{j|K}(\mathbf{x})) \dot{V}_{N_K}(\mathbf{x}, \mathbf{u}) \Big) \\ & + \Big(\sum_{i=j}^{K-1} \big[(1 - \sigma_{j|i}(\mathbf{x})) \dot{V}_{N_i}(\mathbf{x}, \mathbf{u}) \prod_{k=i+1}^{K} \sigma_{j|k}(\mathbf{x}) \big] \Big) \\ & \leq -\gamma(\mu_j \sigma(\mathbf{x}) V_{C_{j|K}}(\mathbf{x}) + (1 - \sigma(\mathbf{x})) V_{N_K}(\mathbf{x})) \Big\} \end{aligned} \tag{A.36}$$

With μ_j as a parameter with default value as $\frac{V_{N_K}(\mathbf{x}_0)}{V_{C_{j|K}}(\mathbf{x}_0)}$, with $\mathbf{x}_0$ equal to the considered initial system state. The set $K_{V_{K+1}}(\mathbf{x})$ is follows the same structure as 2.5.

If $\zeta \to \infty$, according to equations A.32, A.33 and A.34:

$$V(\mathbf{x}) = V_{N_{j-1}}(\mathbf{x}) \quad \mathbf{x} \in \mathcal{S}_j \tag{A.37}$$

$$\mathcal{S}_j = \{\mathbf{x} \in \mathbb{R}^n : V_{N_{j-1}}(\mathbf{x}) \geq tol \cap \dot{V}_{N_{j-1}}(\mathbf{x}, \mathbf{u}) \leq 0 \cap \delta \leq tol_2\} \tag{A.38}$$

And if $\mu = \frac{V_{N_K}(\mathbf{x})}{V_{C_{j|K}}(\mathbf{x})}$ then:

$$K_{V_j}(\mathbf{x}) = \Big\{ \mathbf{u} \in \mathbb{R}^m : \dot{V}_{N_{j-1}}(\mathbf{x}, \mathbf{u}) \leq -\gamma(V_{N_K}(\mathbf{x})) \Big\} \tag{A.39}$$